Research Notes on Transformer architecture deep learning

Transformer architecture deep learning

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

>> Section 1

A decoder consists of an embedding layer, followed by multiple decoder layers, followed by an un-embedding layer. Each decoder consists of three major components: a causally masked self-attention mechanism, a cross-attention mechanism, and a feed-forward neural network. The decoder functions in a similar fashion to the encoder, but an additional attention mechanism is inserted which instead draws relevant information from the encodings generated by the encoders. This mechanism can also be called the encoder–decoder attention. Like the first encoder, the first decoder takes positional information and embeddings of the output sequence as its input, rather than encodings. The transformer must not use the current or future output to predict an output, so the output sequence must be partially masked to prevent this reverse information flow. This allows for autoregressive text generation. For decoding, all-to-all attention is inappropriate, because a token cannot attend to tokens not yet generated. Thus, the self-attention module in the decoder is causally masked. In contrast, the cross-attention mechanism attends to the output vectors of the encoder, which is computed before the decoder starts decoding. Consequently, there is no need for masking in the cross-attention mechanism. Schematically, we have: H ′ = MaskedMultiheadAttention (H , H , H) DecoderLayer (H) = FFN (MultiheadAttention (H ′ , H E , H E)) {\displaystyle {\begin{aligned}H'&={\text{MaskedMultihead Attention}}(H,H,H)\\{\text{DecoderLayer}}(H)&={\text{FFN}} ({\text{MultiheadAttention}}(H',H^{E},H^{E}))\end{aligned}}} where H E {\displaystyle H^{E}} is the matrix with rows being the output vectors from the encoder. The last decoder is followed by a final un-embedding layer to produce the output probabilities over the vocabulary. Then, one of the tokens is sampled according to the probability, and the decoder can be run again to produce the next token, etc., autoregressively generating output text.

>> Section 2

The final points of detail are the residual connections and layer normalization, (denoted as "LayerNorm", or "LN" in the following), which while conceptually unnecessary, are necessary for numerical stability and convergence. The residual connection, which is introduced to avoid vanishing gradient issues and stabilize the training process, can be expressed as follows: y = F(x) + x. The expression indicates that an output y is the sum of the transformation of input x (F(x)) and the input itself (x). Adding the input x can preserve the input information and avoid issues when the gradient of F(x) is close to zero. Similarly to how the feedforward network modules are applied individually to each vector, the LayerNorm is also applied individually to each vector. There are two common conventions in use: the post-LN and the pre-LN convention. In the post-LN convention, the output of each sublayer is L a y e r N o r m (x + S u b l a y e r (x)) {\displaystyle \mathrm {LayerNorm} (x+\mathrm {Sublayer} (x))} where S u b l a y e r (x) {\displaystyle \mathrm {Sublayer} (x)} is the function implemented by the sublayer itself. In the pre-LN convention, the output of each sublayer is x + S u b l a y e r (L a y e r N o r m (x)) {\displaystyle x+\mathrm {Sublayer} (\mathrm {LayerNorm} (x))} The original 2017 transformer used the post-LN convention. It was difficult to train and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018, was found to be easier to train, requiring no warm-up, leading to faster convergence.

>> Section 3

One set of (W Q , W K , W V) {\displaystyle \left(W^{Q},W^{K},W^{V}\right)} matrices is called an attention head, and each layer in a transformer model has multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices, W Q {\displaystyle W^{Q}} and W K {\displaystyle W^{K}} , which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix W V {\displaystyle W^{V}} , in combination with the part of the output projection matrix W O {\displaystyle W^{O}} , determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects. The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers. Concretely, let the multiple attention heads be indexed by i {\displaystyle i} , then we have MultiheadAttention (Q , K , V) = Concat i ∈ [n heads] (

Attention (X W i Q , X W i K , X W i V)) W O {\displaystyle {\text{MultiheadAttention}}(Q,K,V)={\text{Concat}}_{i\in [n_{ \text{heads}}]}({\text{Attention}}(XW_{i}^{Q},XW_{i}^{K},XW _{i}^{V}))W^{O}} where the matrix X {\displaystyle X} is the concatenation of word embeddings, and the matrices W i Q , W i K , W i V {\displaystyle W_{i}^{Q},W_{i}^{K},W_{i}^{V}} are "projection matrices" owned by individual attention head i {\displaystyle i} , and W O {\displaystyle W^{O}} is a final projection matrix owned by the whole multihead attention head. It is theoretically possible for each attention head to have a different head dimension d head {\displaystyle d_{\text{head}}} , but that is rarely the case in practice. As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions: d emb = 768 , n head = 12 , d head = 64 {\displaystyle d_{\text{emb}}=768,n_{\text{head}}=12,d_{\text{head}}=64} Since 12 × 64 = 768 {\displaystyle 12\times 64=768} , its output projection matrix W O ∈ R (12 × 64) × 768 {\displaystyle W^{O}\in \mathbb {R} ^{(12\times 64)\times 768}} is a square matrix.